



CompLit

LEARN. PRACTICE. BUILD. SUCCEED.

CompLit Project Plan



04/20/2026



Team: Origami Inc.

Members: Ronan Fowler,
Cedrick Gutierrez, and Matthew Juhls

Table of Contents

- Introduction..... 3
 - Scope and Purpose of Document (R.F.)..... 3
- System Scope.....3
 - Problem/Opportunity Description (R.F.)..... 3
 - Anticipated Business/Personal Benefits (R.F.)..... 3
 - System Capabilities (C.G.)..... 4
 - System Context Diagram (C.G.)..... 5
- Staff Organization..... 5
 - Team Structure (M.J.)..... 5
- Tracking and control mechanisms (C.G.)..... 6

Introduction

Scope and Purpose of Document (R.F.)

This document details both what our application will contain in terms of features and functionality as well as how we will go about creating it. We will discuss our project scope, reasons for choosing such a scope, our group dynamic and schedule for each individual, and whatever programs/procedures we're using to get this application created.

In short, our application aims to teach computer literacy. The lessons will be taught using various activities and separated into different sections of learning; computer building would be separate from file management for example. We will include lessons for all activities that one might have to learn how to do when using a computer. This even includes things like general lessons on how most photoshop/document programs work. The only thing this wouldn't include is coding, as that's not within our scope although that may be an endeavor for the future of this project.

System Scope

Problem/Opportunity Description (R.F.)

The goal of this project is to teach the average individual how to properly use a computer. As technology advances, many systems and programs are becoming more complicated, but the education for these things doesn't seem to have advanced. Most high schools don't teach tech literacy, for example, and most people won't have the time or money to enroll in some separate class for this kind of thing. People need to be able to know not only how to use a computer, but how to use it in the most efficient way and how to use technological problem solving skills when something does go wrong. You can't always call an IT professional for every little thing.

Anticipated Business/Personal Benefits (R.F.)

We hope to both make money through this application as well as provide a useful and unique service to our users. The best scenario would be a general increase in all of our users' computer capabilities and the saving of money on their part. They should hopefully be able to understand both how to use a computer to the fullest extent as well as how it works by the time they are finished with our application. If not, we hope that the user is able to at least learn what they specifically need from it.

System Capabilities (C.G.)

1. *Text-based Learning*

- Provides step-by-step lessons on computer basics, hardware, and digital literacy for all ages.
- Includes written modules with options like text-to-speech, captions, summaries, adjustable reading speed, zoom, and other interface settings to fit different learning preferences.
- Tracks lesson status (read, completed, needs review) and supports personalized review recommendations.

2. *Video-Based Learning*

- Offers video lessons that parallel the written modules so users can choose or combine formats.
- Embeds short questions, simulated demos, or quick tasks during or after videos to reinforce key concepts.
- Suggests follow-up videos based on user struggles detected through quizzes or simulations.

3. *Interactive Virtual Environment*

- Provides a safe, simulated computer environment where users can practice real tasks without affecting a real system.
- Supports realistic scenarios such as creating an email, cutting/copying/moving files, changing system settings, managing folders, or using basic applications.
- Gives real-time feedback and hints for the current task, helping users correct mistakes as they work.

4. *Virtual Computer Building Simulator*

- Allows users to assemble a virtual desktop from parts such as CPU, GPU, RAM, storage, and motherboard.
- Teaches the role and importance of each component and introduces common installation/troubleshooting issues.
- Simulates system startup, initial setup, BIOS access, and detects basic hardware compatibility problems.

5. *Minigames and Practice Activities*

- Includes games like term-definition matching, hardware identification, timed challenges, and scenario-based challenges.
- Uses an AI-based learning system that adjusts difficulty according to user performance, keeping beginners comfortable and advanced users challenged.
- Provides immediate feedback with a quick explanation for the correct answer, plus an option to see a more detailed explanation when

requested.

6. *Motivation and Progression System*

- Implements points, badges, and progression levels to reward consistent learning and improvement.
- Shows progress dashboards (skills learned, streaks, weak areas) to motivate users and guide what to practice next.

7. *Mixed-Age Interface/Theming and Visual Theme*

- Offers age-based themes such as child, teen, and adult to adjust interface wording, guidance style, color usage, icon emphasis, and content presentation to better match the user's reading and comprehension level
- Maintains the same core content and mechanics while adjusting visuals, language tone, and presentation style.
- Light, dark, high contrast, and sepia themes will preserve readability and navigation clarity, with accessible text contrast and visible focus states maintained in every theme.

8. *Community-Driven Content Expansion*

- Allows verified community educators to submit lessons, practice activities, and scenarios.
- Uses moderators to review and approve community content to maintain quality and accuracy.
- Categorizes all content by lesson type (text, video, simulation, game), difficulty level, and topic area to keep the library organized and easy to search
- Potential for payouts to verified creators, depending on how many uses or traffic a lesson brings, Compliit

System Context Diagram (C.G.)

Our system context diagram is linked on GitHub.

Project Schedule (M.J.)

Our project schedule is linked on GitHub.

Staff Organization

Team Structure (M.J.)

For this project, Ronan has decided that he will be the team leader. To keep the workload evenly distributed, before the due date of each deliverable,

we all pick tasks that we're going to accomplish. Github organization and creation of files will be under Cedrick with Matthew maintaining file documentation in their designated location.

Tracking and control mechanisms (C.G.)

1. Version control and document management

- Store all design, requirements, and specs in a shared document in various online locations such as Google drive/docs and Github
- Use versioning namely dated versions or branches so changes to system capabilities, UI flows, and content structures are tracked and recoverable.
- Require short change logs when updating key documents (what changed, why it changed, who approved it).

2. Regular team meetings

- Regular check-in meetings weekly to review:
 - Progress on each major function (lessons, videos, simulator, minigames, community tools).
 - New issues discovered during development or testing.
 - Proposed changes to requirements or priorities.
- Use these meetings to confirm which tasks are completed, which are in progress, and which are blocked, and to update timelines if needed.
- These meetings will be done between in-person meetings and online through Slack and Discord

3. Task and issue tracking tools

- Use a shared task board trello kanban layout like “Backlog,” “In Progress,” “In Review/Testing,” and “Done.”
- Break the project into small, trackable tasks for example, “Implement lesson tracking,” “Prototype file-management simulation,” “Design age-theme UI,” “Moderation workflow for community lessons”.
- Assign each task to an owner done in our weekly in-person meetings and include due dates, priority, and links back to relevant design/spec documents.